Teaching with numbers: the eight examples, and the three times the machine caught me

Prepared for Michael Lyman · Greenway Marijuana · books-45 · slice C complete

What changed, in one paragraph

You said you wanted the lessons "colorful, interactive and worked examples," and that walls of words are hard to digest. So every lesson on the learning screen can now open with a small table of real numbers instead of four paragraphs of prose: **what you put in, what the system gives back, and why it matters** — with the row that trips people up marked in orange. Eight of them are built so far, out of 82 lessons, and the screen tells you that honestly rather than implying it is finished. The important part is not the eight tables. It is a rule I imposed on myself while building them: **not one number in any example is typed by hand.** Every figure is computed by calling the same engine that runs your actual payroll and your actual penalty calculations, at the moment the page is drawn. That rule caught three genuine errors in a single sitting — three lessons that I had written, read back, and believed were correct, which were teaching you the wrong thing. Those three catches are the real subject of this report, because they are the clearest evidence I can give you that this platform is built differently from the one that burned you.

This also completes **slice C**. All five criteria are done, the branch is merged, and the full suite is at 430 files and 10,379 tests, all passing. You also asked me to clear the three lint warnings we had been carrying rather than leave cosmetic noise in the repo — I did, and **two of the three turned out to be real defects rather than cosmetic**. That is its own section further down, because one of them concerns your mother's distributions directly.

Why a worked example is worth more than a page of explanation

You told me you are a visual learner and that the verbatim law panels were a wall of words. I want to explain why I did not simply shorten the paragraphs, because the fix is not "less text."

An explanation tells you the rule. A worked example tells you the rule **and the shape of the mistake**. Those are different pieces of knowledge, and only the second one changes what you do on a Tuesday.

Here is the difference in practice. I could write: "*Washington's late-payment penalty escalates in steps tied to month-ends.*" That sentence is true, you would nod, and it would not help you, because at no point does it tell you what to actually do when you are two days from a deadline and short on cash. Now look at the same rule as numbers:

You paid	It costs
1 day late	$457.90
16 days late	$457.90
39 days late	$964.00

The instant you see that the first two rows are **the same number**, the rule stops being abstract. Being fifteen extra days late cost you nothing. Being one day later than that — because it crossed the end of February — cost you five hundred dollars. Nobody forgets that after seeing it once, and nobody learns it from the sentence.

That is the whole design. **Given → Output → So what**, with a trap row in orange, sitting above the explanation rather than below it. You lead with the numbers, and read the words only if the numbers surprise you.

The eight examples, and what each one is really about

These are the eight built so far. I have kept the actual computed figures here so you have them on paper as well as on screen.

1. The DOR penalty ladder — the day matters more than the number of days

$4,820.00 owed on a return due 25 January 2027:

	What it costs	The point

When you paid		
1 day late	$457.90 (Step 1)	No grace period at all. One day is the full first step.
16 days late	**$457.90 (Step 1)**	**Trap.** Two weeks later, identical cost. The clock is not counting days.
39 days late	$964.00 (Step 2)	The month turned. Crossing 28 February moved it up a whole step overnight.
157 days late	$1,542.40 (Step 3)	Penalty stops climbing at Step 3. Interest does not.

What to do with this: if a payment is going to be late anyway, getting it in before the last day of the month is worth a full ten percent of the tax. That is a real, actionable decision, and it is invisible in the prose version.

2. Five agencies, one lateness — there is no such thing as "the late penalty"

Exactly $10,000.00 owed to each of five agencies, all paid on the same day:

Agency	Cost
Department of Revenue	**$2,000.00** — trap: the steepest of the five, 9/19/29 percent
Employment Security	$1,200.00
Labor & Industries	$1,200.00
IRS	$1,000.00
Liquor & Cannabis Board	$400.00

Same money, same delay, five different answers — a five-times spread between the cheapest and the most expensive. **When cash is short, the order you pay in is a real decision worth real money.** Note also that L&I and ESD produce the identical $1,200.00, which is exactly why people assume Washington runs one clock. It runs two, and they diverge in other scenarios.

3. The cannabis excise due date — never diary "the 20th"

Sales month	Actually due
January 2027	**22 February** — the 20th is a Saturday
April 2027	20 May — a weekday, so the 20th stands
May 2027	21 June — the 20th is a Sunday

The deadline only ever moves **forward**, never earlier. Diary the computed date, not the rule. And note the system says out loud that it has **not** yet checked holidays — it flags that gap rather than quietly pretending.

4. Eighty hours is not always eighty hours

A budtender at $24.50/hour, 80 hours in a biweekly period:

Roster	Gross
40 + 40	$1,960.00
45 + 35	**$2,021.25** — trap
Difference	**$61.25**

Identical 80 hours, $61.25 apart, because **each workweek stands alone** (29 CFR §778.104, RCW 49.46.130). If you add two weeks together before checking for overtime, you underpay by $61.25 every single period, quietly, until somebody adds it up — and then it is a wage claim with interest, not a rounding difference.

5. Social Security numbers — "accepted" never means "verified"

Input	Resu 4 ga

`078-05-1120`	**Accepted** — trap

That last row is the lesson. These checks prove a number is **impossible**, never that it is **right**. Only the Social Security Administration can do that. A wrong-but-plausible SSN is discovered months later, after the W-2s have gone out.

6. Splitting a salary 26 ways without losing a penny

$55,000.00 across 26 biweekly cheques:

First cheque	**$2,115.50** — the remainder is paid here, at the start
Every other cheque	$2,115.38
All 26 added up	**$55,000.00 exactly**
Same salary monthly	$4,583.37 in January, then $4,583.33

Money is whole cents everywhere in this system. Fractions of a cent do not exist, so every division has to say out loud **who gets the remainder** and prove the pieces add back to the whole. This one matters to you directly — you are paid annually, and your mother's and grandfather's allocations run through the same arithmetic.

7. "Or part thereof" — the three most expensive words on the page

Due 31 January, paid	Counts as
1 February (1 day late)	1 month
28 February (28 days late)	1 month
1 March (29 days late)	**2 months** — trap

Between the 1st and the 28th, being later costs you **nothing extra**. On the 29th it jumps. Same shape as the DOR ladder, arriving from a different statute — which is the point of teaching them together.

8. Somebody starts on Monday — three clocks, counted three different ways

First day 4 January 2027:

Obligation	Due	Counted how
I-9 Section 2	**7 January**	Three **business** days — trap
WA new-hire report	24 January	Twenty **calendar** days
I-9 retention	4 January 2030	Three years from hire, **or** one year after they leave — whichever is later

Three days, twenty days, three years — and the short one counts business days while the middle one counts calendar days. The shortest clock is also the most expensive to miss.

The part that matters most: three times the machine caught me

This is why I am writing you a whole report about eight small tables.

I wrote all eight examples in prose first. I read them back. I believed they were right. Then I made the system compute every number by calling the real engine — and printed what it actually rendered.

Three of the eight were wrong.

The SSN example was backwards

I had written that `078-05-1120` — the famous 1938 wallet-card number — was the one the system refuses, and that `123-45-6789` was accepted. That is the reverse of the truth. The engine refuses `123-45-6789` as a sequential placeholder, and **accepts** `078-05-1120`, because the checks test whether a number is structurally impossible under SSA's randomisation rules, not whether it is famous.

Had I shipped the prose, you would have learned a confident, specific, wrong fact about your own onboarding system — and it would have read perfectly.

The overtime example would have shown a difference of zero

I built the 40+40 versus 45+35 comparison on a function that only calculates straight time. Both rosters would have rendered **$1,960.00**. The example designed to teach you that overtime is weekly would have proved the exact opposite, with real-looking numbers.

Rebuilt on the actual timesheet engine with real clock-in and clock-out punches, it produces the $61.25 difference above.

A row contradicted itself

I wrote that twelve divides $55,000 cleanly, sitting directly beside the engine's output of **$4,583.37**. It does not. The sentence and the number were on the same line, disagreeing.

None of these three was caught by reading it over. All three were caught by printing what the engine actually produced. That is now a standing rule in this repository: never ship a worked example you have not seen rendered with live values. Confidence in the prose is not evidence about the arithmetic.

I am telling you about my own errors deliberately. You have been handed clean-looking output before by a system that was quietly wrong, and the lesson you took from Sage is the right one. The defence is not a smarter author. It is **refusing to let prose and arithmetic come from two different places.**

The guards that keep this true next year

Writing eight correct examples is easy. Keeping them correct after the engines change is the actual problem, and it is the one that bites eighteen months later when nobody is looking.

Automated checks now run on every change, and every one of them was tested by deliberately breaking something and confirming it failed:

- **No hand-typed money.** The checker reads the source code and fails if it finds a currency figure typed as text. A typed `$1,960.00` and a computed one look identical once rendered — so the check reads the source, not the output. It caught a hard-typed `$24.50` I had written beside the variable that actually fed the engine.
- **Every example must show a trap.** An example without one is a demonstration, not a lesson.
- **Every example must be built from the real engines.**
- **No example may ever display a refusal.** If an engine ever declines to compute one of these, the build fails loudly instead of showing you a blank.
- **Sentences are checked against their own numbers.** If a row claims two things "cost the same," the two figures must actually match.

That last one exists because of a mistake worth describing. I ran a mutation test — deliberately sabotaging the code to see whether the tests notice. Nine of ten sabotages were caught. The survivor moved the salary remainder from the first cheque to the last: **every figure on screen changed, and all 34 tests stayed green**, while the sentence went on claiming the remainder lands on the first cheque. The exact kind of drift this module exists to prevent, surviving inside the module built to prevent it.

The fix was not to pin the number. It was to test the **relationship** — the first cheque minus the others equals the remainder, and all 26 pieces sum to the salary — so the same attack cannot come back wearing a different amount. On the retry, all ten were caught.

Honest accounting of what is not done

Eight of 82 lessons have a worked example. The other 74 explain in words only. The screen says so in plain language rather than letting you assume the coverage is complete.

That is a real gap and I am naming it rather than burying it. The eight I chose are the ones where the money or the deadline moves in a way that surprises people. Extending coverage is on the list, and it is straightforward now that the pattern and the guards exist.

Also still open, and visible in the roadmap: the holiday check on due dates, and four teaching modules still not wired to a screen (the §280E/COGS one is next).

You asked about the warnings. Two of them were not cosmetic.

You said you would rather have a clean repo than one carrying cosmetic warnings, and you were right to ask — but the more useful finding is that **they were not cosmetic**. I had been quoting "0 errors, 3 warnings" as a stable baseline for weeks. That phrase is exactly the kind of thing that stops being read once it becomes familiar, and two real defects were sitting inside it.

Here is what each one actually was.

Warning 1 — a type defined twice, which is how two halves of a system start disagreeing

The warning said `PeriodStatus` was imported and unused in a test. Following it, I found that `PeriodStatus` **was defined in two different files** — `ledger-core.ts` and `period-close-core.ts` — with identical values (`open`, `closed`, `locked`) and **no connection between them whatsoever**.

Why that matters, in your terms. A period status is what tells the system whether a month can still be posted into. `period-close-core` owns the real one: it mirrors the constraint in your actual database and carries the plain-English meaning of each status, including that **locked** means a tax return has been filed on those numbers and nothing reopens it. The second copy was a silent twin. If someone later added a fourth status to one copy — say `under_review` — the code would compile perfectly and the database would reject it at runtime, and the error would surface somewhere far away from the cause.

I checked before deleting: the `ledger-core` copy was referenced by **nothing at all** — no other file, no lesson, not even the file it lived in. A pure orphan. It is gone, with a note left where it stood explaining where the real one lives.

Warning 2 — a function that was taught but never directly tested

This is the one I would have been most annoyed to discover later. The warning flagged `assessProportionality` as imported and unused. That function is **taught in one of your lessons**, so it is not obscure — and it is the one that examines whether your distributions match your ownership percentages.

That function matters to you specifically. Your ownership is 85% you, 10% your mother, 5% your grandfather — and your mother's share is **allocated but not paid**. So your distributions are, by design, not proportionate, and this is the function that has to describe that situation accurately without overstating it.

It *was* being tested, but only indirectly, through the wrapper that runs a whole year. That wrapper validates the shareholder list before passing it on, which means it can **never** hand the function an empty list or a single shareholder. Those two edges were untestable by that route and untested in fact.

The tempting fix was to delete the import. That would have produced a clean lint run and **less coverage** — the warning gone and the gap it marked now invisible. I added three direct tests instead. One of them is your real pattern, priced by the engine:

Shareholder	Pro rata share	Actually paid	Difference
Michael (85%)	$44,625.00	$50,000.00	**over by $5,375.00**
Mother (10%)	$5,250.00	$0.00	**under by $5,250.00**
Grandfather (5%)	$2,625.00	$2,500.00	under by $125.00

The differences net to exactly zero, which is the arithmetic check that nothing was invented or lost. And the engine is careful about what it concludes: an unequal distribution is **not**, by itself, a second class of stock, because that test turns on the rights in your governing documents rather than on what the cheques happened to be. What it does insist on is that the gap gets **characterised** as something — a loan from the company, additional compensation, or a gift between shareholders. Leaving it uncharacterised is the only genuinely bad option. That is a real conversation for you and your mother, and it is now on the record with a number attached.

Both new tests were then deliberately sabotaged to confirm they actually fail when the engine is wrong. They did. The engine file was restored byte-for-byte afterwards.

Warning 3 — genuinely just redundant text

The third was a type imported into a test that used the runtime list instead. Verified by reading every reference, then removed. This one really was cosmetic — but I only know that because I checked, and I could not have known it from the warning text alone.

What the trail led to: fifteen more duplicates

If one type was silently duplicated, the obvious question is how many others are. I scanned every single-line type definition in the codebase and found **sixteen defined identically in two or more files.** Two are genuinely concerning:

- `AccountType` — defined in **three** accounting modules. This is the list of what an account can be: asset, liability, equity, income, COGS, expense. Three copies.
- `SourceKind` — defined twice. This is what a journal entry came from: a POS sale, payroll, excise, an ATM, and so on.

Both are long lists, and a value added to one copy and not the others compiles cleanly while two halves of your ledger quietly disagree about what an account is. **I have not fixed these**, and I want to be direct about why: each needs its owning module chosen and every other reference repointed. That is a piece of work in its own right, and half-doing it across your accounting core would be worse than naming it. It is written into the roadmap as item R2 with all sixteen listed.

One more thing worth telling you, because it is the kind of thing I would want told to me. **My first version of that scan was wrong.** It read each definition only up to the first semicolon, so any type spanning several lines got cut off at its opening brace, and any two that happened to start the same way compared as identical. It confidently reported **36** duplicates. Several were inventions of my own bad measurement. I rewrote it to skip anything it could not read in full, and the honest number is **16** — and even that is a floor rather than a total, because the multi-line types remain unmeasured.

I am telling you the wrong number as well as the right one because a count I hand you is only worth anything if it survives being checked, and this one did not the first time.

The result: the repository now reports zero errors and zero warnings. Confirmed in the real build, not just on my machine.

Where we are, and what is next

Slice C is complete. All five criteria closed. Alongside the examples, this change also mirrored three Washington statutes locally so their quotes can be checked word-for-word against the real law, fixed two genuine misquotes in our own citations, and retired a duplicate percentage-calculation routine so two slightly different versions can no longer both be right.

Every quoted statute in the platform — 306 of them — is now verified character-by-character against the actual source text, with zero failures.

The agreed order from here:

1. **Slice A — W-2 and W-3.** Next up. Boxes 1 through 6, box 12 codes, the state boxes, and the reconciliation that proves your four quarterly 941s agree with your W-3. Box 17 stays blank — Washington has no income tax.
2. **Slice D** — the Form / Why / Check tab system.
3. **Slice B** — K-1, 1120-S and 1040. Blocked until you send Form 7203.
4. **Slice E** — the DOR Combined Excise Tax Return preview you asked for. Blocked until you send your filed July return.

What I still need from you

The time-critical one first:

- **The twelve 2027 rate notices** — this is the one with a deadline attached, since your first payroll is 1 January 2027.
- Your **filed July DOR return** (unblocks slice E).
- **Form 7203**, **Form 2553 with the CP261**, ending AAA, and the S-election tax year (unblocks slice B).
- Q3/Q4 2026 returns, the §6699(e) amount, quarterly federal short-term rates, the depreciation schedule.

Nothing in this platform will invent any of those. Where a number is missing, the system refuses and tells you which document unblocks it — which is exactly what it did four times while I was building these eight examples.

Verified before merge: 430 test files, **10,379 tests**, all passing · type-check clean · **lint at zero errors and zero warnings** · 306 statute quotes verified with zero failures · 10 of 10 sabotage tests caught on the worked examples, plus 2 of 2 on the new proportionality tests.